```
//     Ł       u       k       a       s       z
    . "\x01\x41\x00\x75\x00\x6b\x00\x61\x00\x73\x00\x7a";
//                                  ł
var_dump(mb_stristr($utf16name, "\x01\x42", false, 'UTF-16'));
```

```
Output:
string(12) "Aukasz"
```

Notice the output? It's what you'll see when running the code on a UTF-8-based system and shows what happens when UTF-16 is treated like it is UTF-8. *0x01* is an ASCII (and therefore UTF-8) non-printing control character, *0x41* is capital "A", then the rest of the return string alternates between *0x00* (null byte) and ASCII/UTF-8 letters "u", "k", "a", and so on. A nice demonstration of the point I'm trying to hammer home about how text really works!

It's also worth mentioning here that pretty much all `mbstring` functions work internally only with UTF-8 text. Using them with any other multi-byte encoding will (when the non-UTF-8 encoding is specified in the function arguments) trigger internal conversion to UTF-8, and any string data being returned is converted back to its original encoding for output. This makes the `mbstring` functions significantly slower (and consume more memory) than their core PHP counterparts, and therefore worth thinking about if handling large amounts of text. We'll revisit the internals – and resulting performance impact – of the `mbstring` module later in the chapter. For now, here's something to think about:

```
$res = mb_strstr($utf16name, "\x01\x41\x00\x75", false, 'UTF-16');
var_dump(bin2hex($res));

$res = mb_strstr($utf16name, "\x01\x41\x00\x75");
var_dump(bin2hex($res));

$res = strstr($utf16name, "\x01\x41\x00\x75");
var_dump(bin2hex($res));
```

```
Output:
string(24) "01410075006b00610073007a"
string(24) "01410075006b00610073007a"
string(24) "01410075006b00610073007a"
```